



UTPL
UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

INFORME TEORICO/PRACTICO

FACULTAD DE INGENIERÍAS Y ARQUITECTURA
CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

PROGRAMCION ORIENTADA A OBJETOS

Docente: Bustamente Granda Wayner Xavier

Autor: Juan Pablo Loja

Periodo: Abril 2026 - Agosto 2026

Loja - Ecuador

Informe teórico/práctico de POO en Python

La Programación Orientada a Objetos (POO) en Python nos permite tener una organización en nuestros códigos mediante clases y objetos, facilitando la reutilización, mantenimiento y modularidad de los programas, Aunque Python y Java comparten los mismos conceptos principales de la POO, la sintaxis y forma de implementación presentan algunas diferencias importantes.

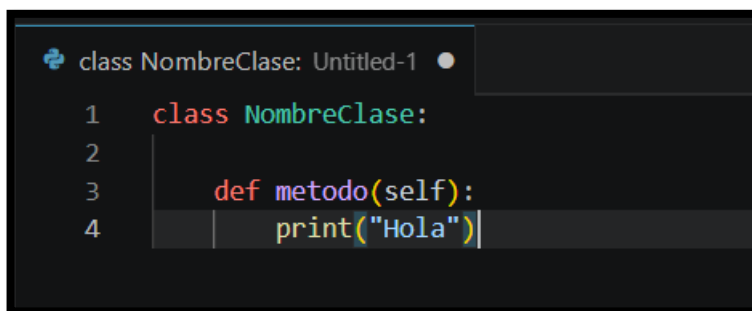
A continuación, se mostrará la implementación de clases, atributos, métodos, constructores, toString(), niveles de acceso, etc. en **Python**.

1. Implementación

1.1 Clases en Python

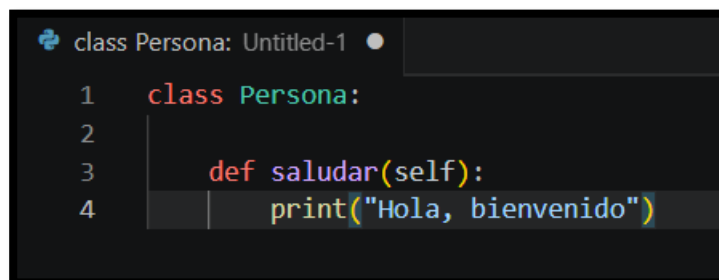
Una clase es una plantilla o modelo que define las características (atributos) y comportamientos (métodos) que tendrán los objetos.

- **Estructura básica de una clase:**



```
class NombreClase:
1  class NombreClase:
2
3      def metodo(self):
4          print("Hola")
```

- **Ejemplo práctico:**



```
class Persona:
1  class Persona:
2
3      def saludar(self):
4          print("Hola, bienvenido")
```

Podemos entender lo siguiente:

- La palabra **“class”** es la palabra reservada para crear clases.
- La palabra **“Persona”** es el nombre que se le asigna a la clase.
- **“def”** es la definición de métodos.
- **“self”** es la cual representa el objeto actual.

Finalmente, se imprime el resultado de **“Hola, bienvenido”**

1.2 Atributos en Python

Existen dos tipos de atributos en Python: los atributos de clase los cuales están definidos directamente en el cuerpo de la clase compartidos por todas las instancias y los atributos de instancia los cuales son definidos dentro del constructor **“__init__”** son variables que pertenecen a una clase u objeto.

- **Ejemplo práctico:**

```
class Persona: Untitled-1
1  class Persona:
2
3      def __init__(self):
4          self.nombre = "Juan"
5          self.edad = 20
```

Básicamente los atributos son estas variables como “**nombre**” y “**edad**” las cuales se les otorgan un valor como “**Juan**” y “**20**” estas pertenecen a la clase “**Persona**”.

Ademas se usa el “**self**” el cual nos permite que los atributos pertenezcan al objeto.

1.3 Métodos en Python

Los métodos son funciones que pertenecen a una clase. Siempre reciben “**self**” como primer parámetro (equivalente al **this** de java)

- **Ejemplo práctico:**

```
class Persona: Untitled-1
1  class Persona:
2
3      def saludar(self):
4          print("Hola")
```

El método “**saludar()**” define una acción que puede realizar el objeto.

1.4 Constructores en Python

El constructor en Python es el método especial “**__init__**”, Se ejecuta automáticamente al instanciar la clase y recibe “**self**” como primer parámetro (referencia al objeto actual) equivalente igualmente al “**this**” en Java.

- **Ejemplo práctico:**

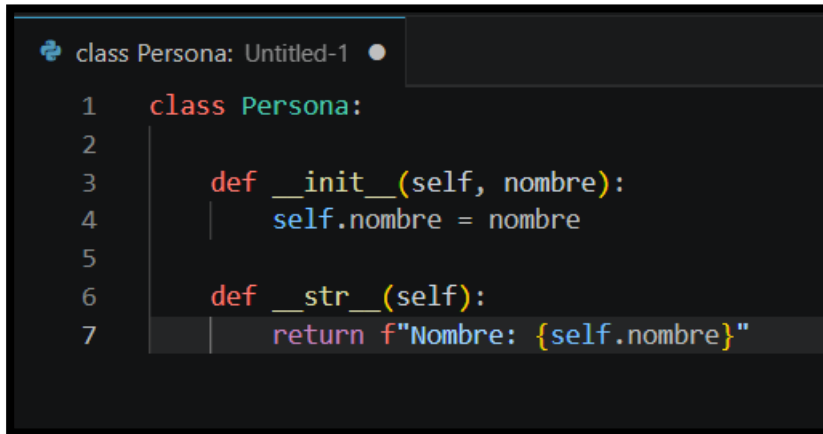
```
class Persona: Untitled-1
1  class Persona:
2
3      def __init__(self, nombre, edad):
4          self.nombre = nombre
5          self.edad = edad
```

Como se explicó este método se ejecuta automáticamente al crear un objeto. Cuando se crea un objeto, el constructor inicializa sus atributos.

1.5 toString() en Python

En Python, el equivalente al método toString() de Java es “`__str__()`”. Se invoca automáticamente cuando se usa print() o str(). También existe “`__repr__()`” para representación técnica del objeto.

- **Ejemplo práctico:**



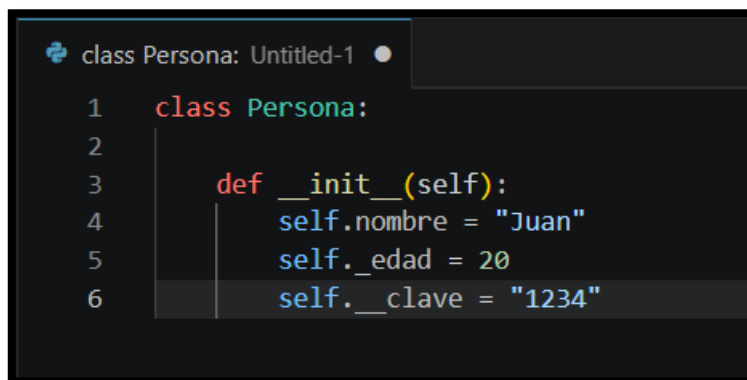
```
class Persona:
1  class Persona:
2
3      def __init__(self, nombre):
4          self.nombre = nombre
5
6      def __str__(self):
7          return f"Nombre: {self.nombre}"
```

Este ejemplo muestra como el método “`__str__()`” devuelve una representación en texto del objeto.

1.6 Niveles de acceso en Python

En Python no existen modificadores de acceso formales como en Java existe el “public, private, protected”. En lugar de eso usa convenciones de nomenclatura.

- **Ejemplo práctico:**



```
class Persona:
1  class Persona:
2
3      def __init__(self):
4          self.nombre = "Juan"
5          self._edad = 20
6          self.__clave = "1234"
```

Las convenciones se las interpreta de la siguiente manera:

- public es “**nombre**” que puede ser accesible desde cualquier lugar.
- protected es “**_edad**” es para el uso interno recomendado.
- private “**__clave**” tiene un acceso restringido desde afuera.

Python implementa la Programación orientada a objetos de forma mas flexible y sencilla que Java, utilizando una sintaxis menos estricta y mas legible. A pesar de unas

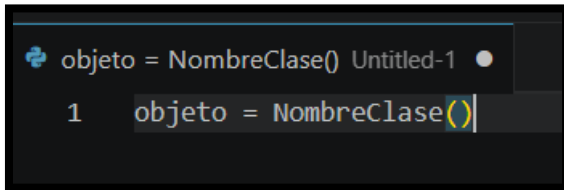
cuentas diferencias, ambos lenguajes de programación comparten conceptos fundamentales como clases, objetos, atributos, métodos y constructores.

2. Ejecutar, invocar e instanciar una clase mediante su objeto

2.1 Instanciar un objeto en Python

Instanciar significa crear un objeto a partir de una clase, la clase funciona como un molde, mientras que el objeto es la instancia creada con ese molde

- **Estructura básica:**



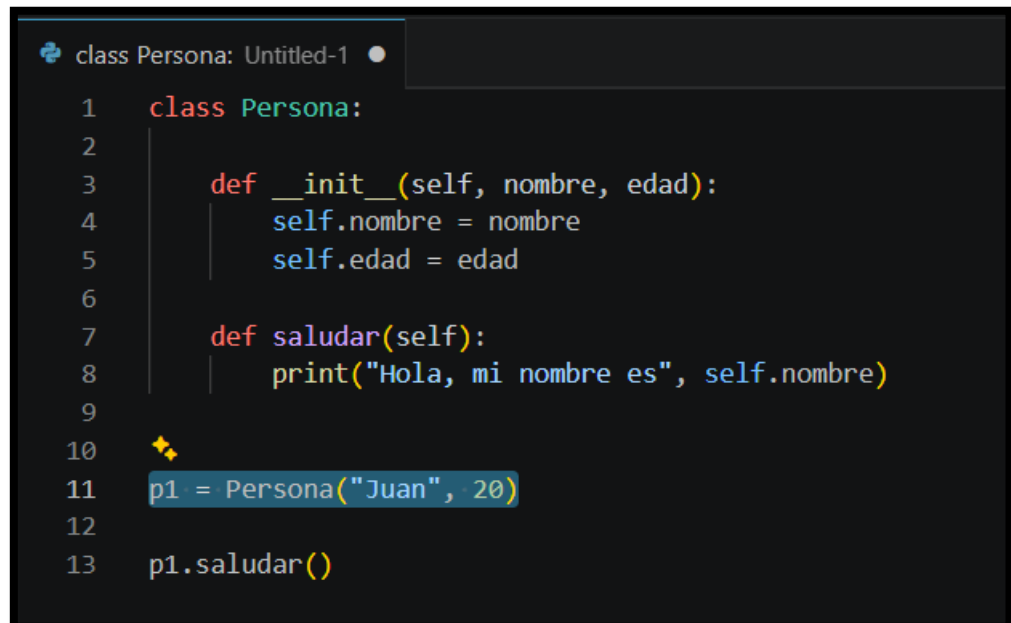
```
objeto = NombreClase() Untitled-1
1 objeto = NombreClase()
```

Interpretamos de la siguiente manera:

“objeto” es el nombre que se le designa a la variable que almacenara el objeto

“NombreClase()” es la cual llama al constructor de la clase.

- **Ejemplo práctico:**



```
class Persona: Untitled-1
1 class Persona:
2
3     def __init__(self, nombre, edad):
4         self.nombre = nombre
5         self.edad = edad
6
7     def saludar(self):
8         print("Hola, mi nombre es", self.nombre)
9
10
11 p1 = Persona("Juan", 20)
12
13 p1.saludar()
```

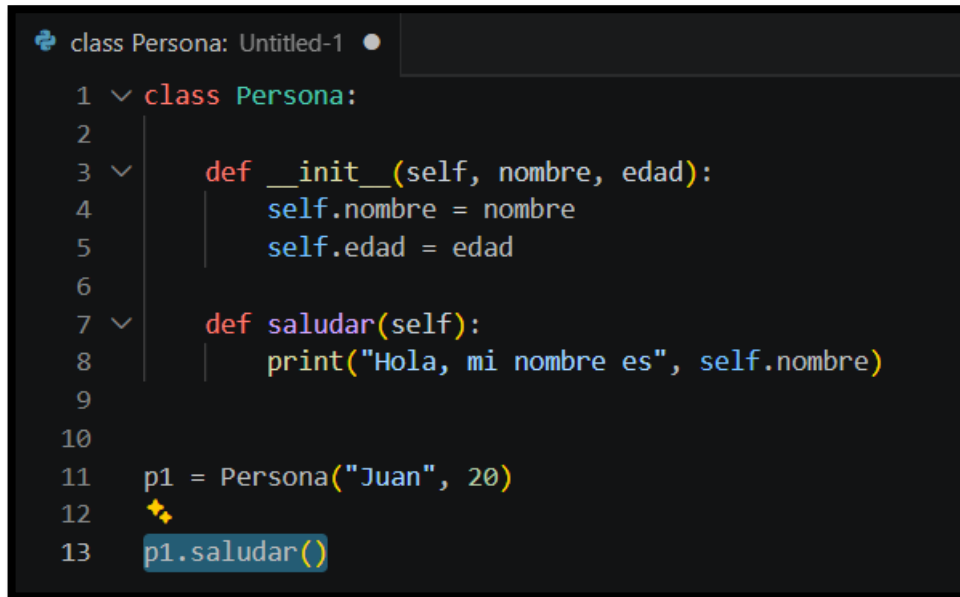
En este ejemplo

- “class Persona” es donde se crea la clase, de ahí el constructor.
- “__init__()” es el que inicializa los atributos.
- “self.nombre” es donde se guarda el nombre del objeto,
- “p1 = Persona("Juan", 20)” aquí es donde instancia el objeto.
- “p1.saludar()” y por último aquí es donde se invoca el método.

2.2 Invocar en Python

Invocar en Python significa llamar o ejecutar un método mediante un objeto.

- Tomaremos el ejemplo anterior:



```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
    def saludar(self):
        print("Hola, mi nombre es", self.nombre)

p1 = Persona("Juan", 20)
p1.saludar()
```

Explicando específicamente la parte donde ocurre es en lo señalado:

- “p1” es el objeto.
- “saludar()” es el método invocado.

2.3 Ejecución en Python

Para la ejecución de un código en Python con POO podemos usar diferentes IDE como PyCharm, VS Code u otro online como Google Colaboratory o Replit. Python interpretará la clase y ejecutará las instrucciones del objeto. Para una mejor comprensión vamos a crear un código completo de ejemplo comprobable. Se podrá encontrar de Google Colaboratory.

En Python, los objetos se crean mediante la instanciación de clases. A través de estos es posible acceder a atributos y ejecutar métodos definidos dentro de la clase. Este mecanismo nos permite aplicar los principios de la Programación Orientada a objetos de forma organizada y reutilizable.

3. **Código de ejemplo comprobable online vía: Google Colaboratory o Replit, o la opción que crea más factible. (en este apartado deberá colocar la URL compartida con privilegios)**

- <https://colab.research.google.com/drive/1G4ObdaHN1ie4CjJ6xRRI1Fpj5MvapJSm?usp=sharing>

4. Tabla comparativa de semejanzas, diferencias, ventajas, desventajas, etc. de trabajar POO en Java y/o Python.

Semejanzas entre Java y Python en POO	
Java	Python
Soporta clases, objetos, atributos y métodos	Soporta clases, objetos, atributos y métodos
Implementa los 4 pilares de POO: encapsulamiento, herencia, polimorfismo, abstracción	Implementa los 4 pilares de POO: encapsulamiento, herencia, polimorfismo, abstracción
Permite interfaces / clases abstractas	Permite clases abstractas (módulo abc)
Constructores para inicializar objetos	Constructor <code>init</code> para inicializar objetos
Soporte para métodos estáticos y de clase	Soporte para <code>@staticmethod</code> y <code>@classmethod</code>
Tipado fuerte (no mezcla tipos arbitrariamente)	Tipado fuerte (no mezcla tipos arbitrariamente)

Aspecto	Ventajas de Java	Ventajas de Python
Rendimiento	Alto rendimiento gracias al compilador JIT de la JVM, que optimiza el bytecode en tiempo real logrando velocidades cercanas al código nativo.	Productividad alta: sin declarar tipos, sin compilar, sin boilerplate obligatorio. El prototipado es mucho más rápido.
Tipado	Tipado estático: los errores de tipo se detectan en tiempo de compilación, antes de ejecutar. Reduce bugs en producción.	Tipado dinámico: no es necesario declarar tipos, lo que acelera el desarrollo. Admite type hints opcionales.
Sintaxis / Legibilidad	Estructura clara y formal. La verbosidad obliga a definir todo explícitamente, lo que facilita la lectura en proyectos grandes.	Sintaxis simple y limpia, muy cercana al lenguaje humano. Menos líneas para lograr el mismo resultado que Java.
Portabilidad	'Write once, run anywhere': el mismo .jar corre en cualquier sistema operativo con JVM instalada sin recompilar.	Multiplataforma: corre en Windows, macOS, Linux e incluso microcontroladores (MicroPython) sin recompilación.
Ecosistema	Ecosistema maduro: Spring, Hibernate, Maven/Gradle y miles de librerías estables con décadas de soporte empresarial.	Ecosistema de datos e IA inigualable: NumPy, Pandas, TensorFlow, PyTorch, scikit-learn son referentes mundiales.
Seguridad	Gestión de memoria automática, sin punteros directos y modelo de seguridad consolidado para entornos empresariales.	Curva de aprendizaje suave: lenguaje más recomendado para principiantes por su legibilidad y ausencia de elementos de bajo nivel.
Escalabilidad	Arquitectura pensada para sistemas grandes y distribuidos: microservicios, servidores empresariales, Android.	Herencia múltiple nativa: soporta heredar de varias clases simultáneamente, algo que Java no permite directamente.

Aspecto	Desventajas de Java	Desventajas de Python
Sintaxis / Verbosidad	Código muy verboso: getters, setters, imports, public static void main... El mismo programa ocupa más líneas que en Python.	Tipado dinámico en runtime: los errores de tipo aparecen al ejecutar, no antes. En proyectos grandes esto genera bugs difíciles de rastrear.
Rendimiento / Velocidad	Arranque lento: la JVM tarda en inicializarse, lo que afecta funciones serverless y scripts cortos.	Menor rendimiento: al ser interpretado, es considerablemente más lento que Java en tareas de cómputo intensivo.
Curva de aprendizaje	Curva pronunciada: conceptos como genéricos, interfaces, anotaciones y el sistema de tipos puede abrumar a principiantes.	GIL (Global Interpreter Lock): CPython solo ejecuta un hilo real a la vez. La concurrencia multi-núcleo requiere workarounds.
Herencia	Sin herencia múltiple directa: solo permite heredar de una clase. Las interfaces compensan, pero agregan complejidad.	Alto consumo de memoria: los objetos Python consumen más RAM que sus equivalentes en Java debido al overhead del intérprete.
Ecosistema / Área	Pobre en ciencia de datos: aunque existen librerías, no tiene el ecosistema de Python para ML, IA o análisis de datos.	Débil en móvil: no hay soporte nativo para desarrollo Android/iOS, a diferencia de Java/Kotlin.

Aspecto	Diferencias en Java	Diferencias en Python
Tipo de lenguaje	Compilado a bytecode (JVM)	Interpretado (CPython)
Tipado	Estático — se declara el tipo explícitamente	Dinámico — el tipo se infiere en ejecución
Sintaxis	Llaves { }, punto y coma ; obligatorios	Indentación significativa, sin llaves ni ;
Constructor	public NombreClase(params) { }	def __init__(self, params):
Niveles de acceso	public / private / protected (forzado)	Convención: _ (protegido) y __ (privado)
Herencia	Simple con extends; múltiple via interfaces	Múltiple nativa: class Hijo(A, B):
toString()	@Override public String toString() { }	def __str__(self): / def repr(self):
Getters / Setters	Métodos get/set manuales o con Lombok	Decorador @property integrado
Método estático	Palabra clave static	@staticmethod o @classmethod
Polimorfismo	Explícito con @Override / Overload	Duck typing implícito por diseño
Rendimiento	Alto — compilador JIT de la JVM	Menor — interpretado en tiempo real
Concurrencia	Multi-hilo nativo sin limitaciones	GIL limita el paralelismo real en CPU
Gestión de memoria	Garbage Collector de la JVM	GC + conteo de referencias (CPython)
Uso principal	Empresarial, Android, backend corporativo	IA, ciencia de datos, scripting, web

Tanto en Java como en Python que son lenguajes orientados a objetos que comparten principios fundamentales de programación. Sin embargo, presentan algunas diferencias importantes como en la sintaxis, ejecución y nivel de complejidad. Python tal vez puede destacar por su simplicidad rapidez a la hora del desarrollo, mientras que Java puede sobresalir por su robustez, rendimiento y estructura mas estricta en los proyecto que son de gran escala.